DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Ingeniería de Software

Análisis y Diseño de Software - NRC 30724

Taller:

Guía 2

Casos de uso, clases y secuencia CRUD

Grupo

Saraguro Leidy

Ñacato Alexander

Jaguaco Jonathan

Obando Erick

Instructor: Ing. Jenny Alexandra Ruiz Robalino

Fecha: 28/04/2026

TALLER 4 UNIDAD 1	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

1. Objetivo

1.1.

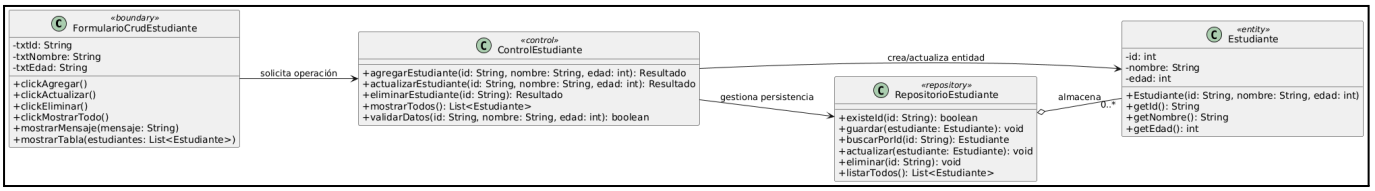
2. Desarrollo

2.1. Diagrama de clases de análisis

```
@startuml
 skinparam classAttributeIconSize 0
 skinparam shadowing false
 left to right direction

 class "FormularioCrudEstudiante" <<boundary>> {
   - txtId: int
   - txtNombre: String
   - txtEdad: int
   + clickAgregar()
   + clickActualizar()
   + clickEliminar()
   + clickMostrarTodo()
   + mostrarMensaje(mensaje: String)
   + mostrarTabla(estudiantes: List<Estudiante>)
 }

 class "ControlEstudiante" <<control>> {
   + agregarEstudiante(id: int, nombre: String, edad: int): Resultado
   + actualizarEstudiante(id: int, nombre: String, edad: int): Resultado
   + eliminarEstudiante(id: int): Resultado
   + mostrarTodos(): List<Estudiante>
   + validarDatos(id: int, nombre: String, edad: int): boolean
 }

 class "Estudiante" <<entity>> {
   - id: int
   - nombre: String
   - edad: int
   + Estudiante(id: int, nombre: String, edad: int)
   + getId(): int
   + getNombre(): String
   + getEdad(): int
 }

 class "RepositorioEstudiante" <<repository>> {
   + existeId(id: int): boolean
   + guardar(estudiante: Estudiante): void
   + buscarPorId(id: int): Estudiante
   + actualizar(estudiante: Estudiante): void
   + eliminar(id: int): void
   + listarTodos(): List<Estudiante>
```

TALLER 4 **UNIDAD 1**	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

```
}

FormularioCrudEstudiante --> ControlEstudiante : solicita operación
ControlEstudiante --> Estudiante : crea/actualiza entidad
ControlEstudiante --> RepositorioEstudiante : gestiona persistencia
RepositorioEstudiante o-- "0..*" Estudiante : almacena
@enduml
```



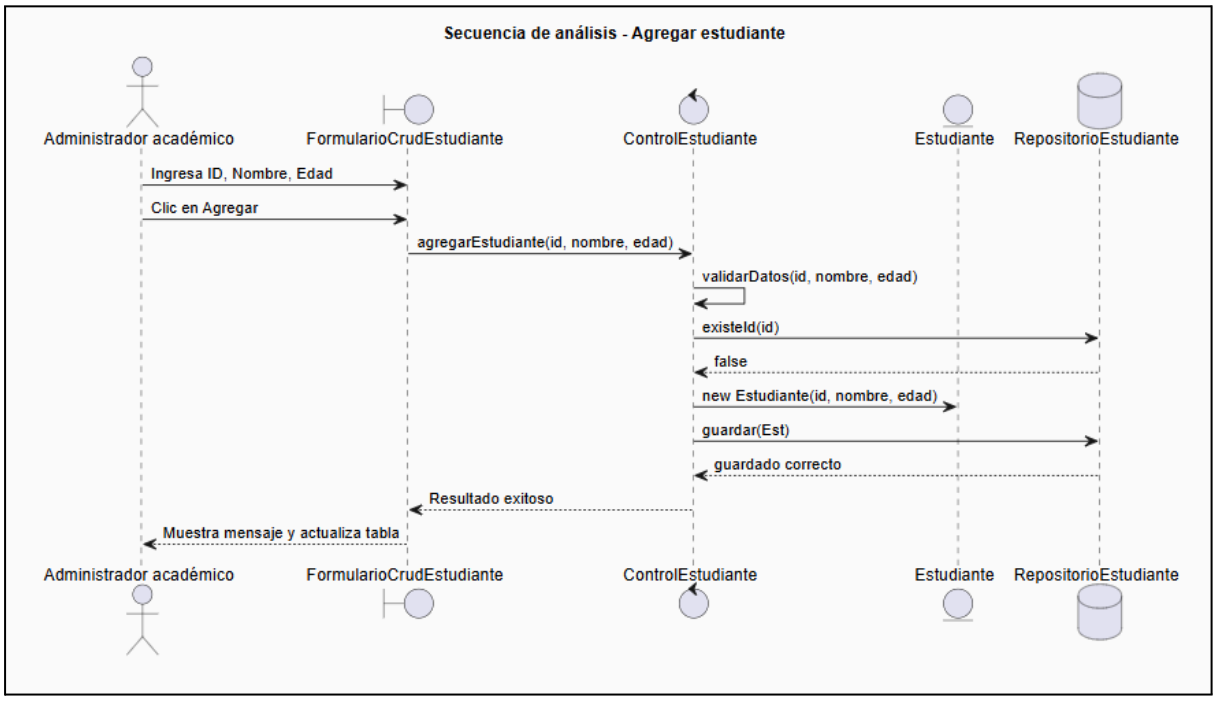
Especificación breve:

2.2. Diagramas de secuencia

2.2.1. Secuencia: Agregar estudiante

```
@startuml
title Secuencia de análisis - Agregar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
entity "Estudiante" as Est
database "RepositorioEstudiante" as Repo

Admin -> UI : Ingresa ID, Nombre, Edad
Admin -> UI : Clic en Agregar
UI -> Ctrl : agregarEstudiante(id, nombre, edad)
Ctrl -> Ctrl : validarDatos(id, nombre, edad)
Ctrl -> Repo : existeId(id)
Repo --> Ctrl : false
Ctrl -> Est : new Estudiante(id, nombre, edad)
Ctrl -> Repo : guardar(Est)
Repo --> Ctrl : guardado correcto
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y actualiza tabla
@enduml
```

TALLER 4 **UNIDAD 1**	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick



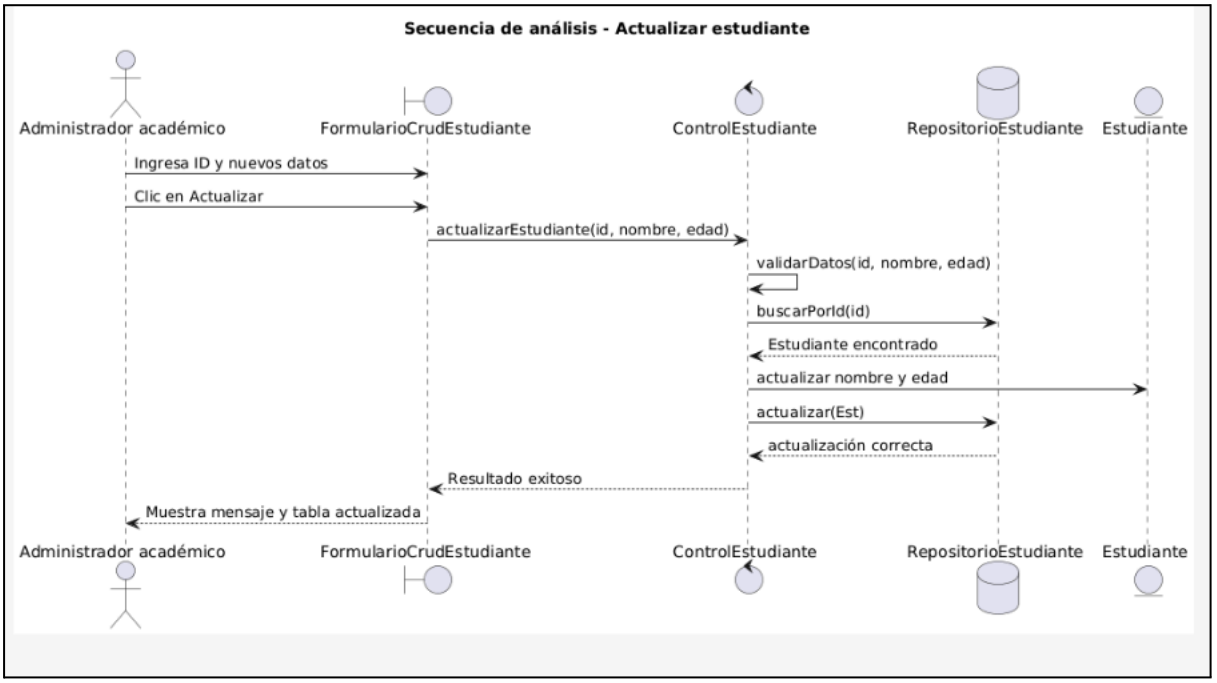
Especificación breve:

2.2.2. Secuencia: Actualizar estudiante

```
@startuml
title Secuencia de análisis - Actualizar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo
entity "Estudiante" as Est

Admin -> UI : Ingresa ID y nuevos datos
Admin -> UI : Clic en Actualizar
UI -> Ctrl : actualizarEstudiante(id, nombre, edad)
Ctrl -> Ctrl : validarDatos(id, nombre, edad)
Ctrl -> Repo : buscarPorId(id)
```

TALLER 4 UNIDAD 1	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

```
Repo --> Ctrl : Estudiante encontrado
Ctrl -> Est : actualizar nombre y edad
Ctrl -> Repo : actualizar(Est)
Repo --> Ctrl : actualización correcta
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y tabla actualizada
@enduml
```



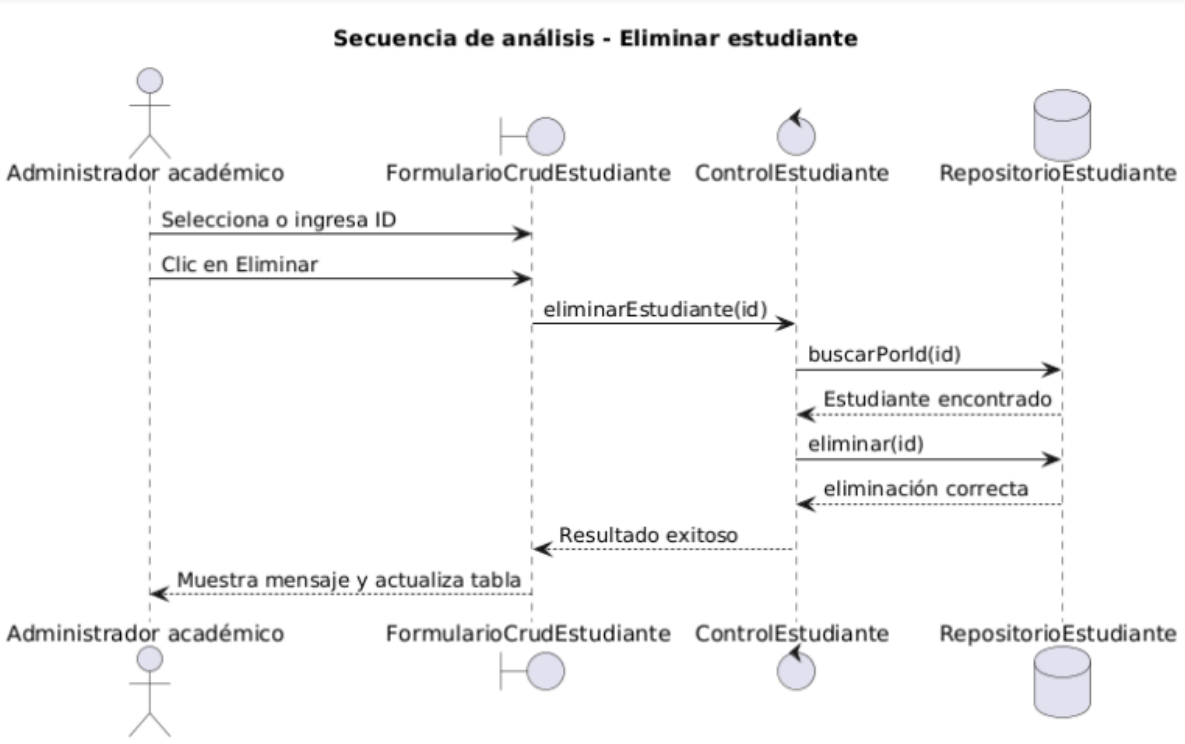
Especificación breve:

2.2.3. Secuencia: Eliminar estudiante

```
@startuml
title Secuencia de análisis - Eliminar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo
```

TALLER 4 UNIDAD 1	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

```
Admin -> UI : Selecciona o ingresa ID
Admin -> UI : Clic en Eliminar
UI -> Ctrl : eliminarEstudiante(id)
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Estudiante encontrado
Ctrl -> Repo : eliminar(id)
Repo --> Ctrl : eliminación correcta
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y actualiza tabla
@enduml
```

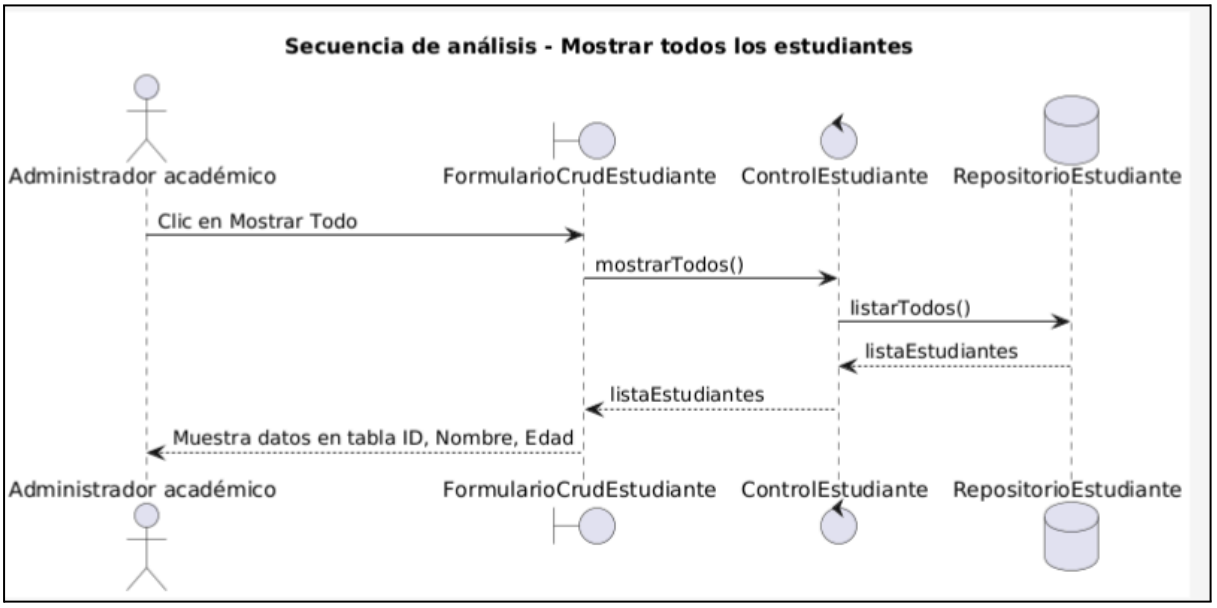


Especificación breve:

2.2.4. Secuencia: Mostrar todos los estudiantes

TALLER 4 **UNIDAD 1**	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

```
@startuml
title Secuencia de análisis - Mostrar todos los estudiantes
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo

Admin -> UI : Clic en Mostrar Todo
UI -> Ctrl : mostrarTodos()
Ctrl -> Repo : listarTodos()
Repo --> Ctrl : listaEstudiantes
Ctrl --> UI : listaEstudiantes
UI --> Admin : Muestra datos en tabla ID, Nombre, Edad
@enduml
```



Especificación breve:

2.3. Trazabilidad

TALLER 4 **UNIDAD 1**	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

RF	Caso de uso	Actor	Prioridad	Criterio de aceptación	Evidencia PlantUML
RF-01	Agregar estudiante	Administrador académico	Alta	El sistema guarda un estudiante con ID único,Nombre obligatorio y Edad válida.	UC1
RF-02	Actualizar estudiante	Administrador académico	Alta	El sistema modifica Nombre y/o Edad luego de validar campos obligatorios	UC2
RF-03	Eliminar estudiante	Administrador académico	Media	El sistema elimina o marca como eliminado al estudiante seleccionado.	UC3
RF-04	Mostrar todo	Administrador académico	Alta	El sistema muestra una tabla con las columnas ID, Nombre y Edad.	UC4
RF-05	Validar datos del estudiante	Sistema	Alta	El sistema impide guardar o actualizar registros con ID vacío, nombre vacío o edad inválida.	UC5

3. Conclusiones

(colocar la experiencia de nosotros al implementar esta guia)

TALLER 4 UNIDAD 1	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

4. Recomendaciones

(colocar lo que se reviso en el plantUML, el error en el tipo de dato edad)